



VR Electric Vehicle

An immersive VR training simulation that teaches battery diagnostic procedures for electric vehicles — from safety gear to OBD-II data interpretation.

Built by **Ziber Art Studio** · July 2026



Unity 2022.3 LTS



XR Interaction Toolkit 2.3.2



Oculus + OpenXR

6

DIAGNOSTIC STEPS

20+

C# SCRIPTS

2

TARGET PLATFORMS

10

CORE SYSTEMS



Table of Contents

1 Project Overview

2 Target Platforms & Tech Stack

3 Scene Flow & Architecture

4 The 6-Step Diagnostic Procedure

5 Script Reference

6 Core Systems

7 3D Assets & Resources

8 Project Folder Structure

9 Build & Deployment

1

Project Overview

VR Electric Vehicle is an immersive VR training application that teaches users how to perform battery diagnostics on electric vehicles. Designed for **Meta Quest** and **Windows Standalone** VR headsets, it walks trainees through a complete diagnostic workflow — from putting on safety equipment to interpreting OBD-II scanner data and answering condition-assessment questions.

The project was built by **Ziber Art Studio** using a linear, guided-procedure model: each of the six steps must be completed before advancing, with real-time task tracking, visual feedback (checkmarks, highlights, UI panels), and a final scored results clipboard.



Guided Learning

Step-by-step instructions with arrow hints, toggle groups & hover tooltips



Interactive VR

Grab gloves, glasses & OBD scanner; trigger zones & 3D UI toggles



Immediate Feedback

Checkmarks on task completion; instant correct/incorrect quiz results



Result Compilation

All task outcomes displayed on a final clipboard for review

2

Target Platforms & Tech Stack

 **Target Platforms**

PLATFORM	BACKEND	VR RUNTIME
Meta Quest (Android)	IL2CPP · ARM64	Oculus XR + OpenXR
Windows Standalone	Mono · x86_64	Oculus / OpenXR

 **Technology Stack**

Engine	Unity 2022.3 LTS
Rendering	Universal Render Pipeline (URP 14.x) · Linear color space
VR Framework	XR Interaction Toolkit 2.3.2
VR Plugins	Oculus XR Plugin 4.0.0 · OpenXR Plugin 1.7.0
UI System	Unity UI (uGUI) + TextMesh Pro 3.0.6
Input	Unity Input System (New) · <code>activeInputHandler: 2</code>

Scripting

C# — IL2CPP (Android) / Mono (Standalone)

Template`com.unity.template.urp-blank`**Third-party**

QuickOutline (Chris Nolet) · Oculus Hands animation · Kenney Audio (CC0)

Testing

Unity Test Framework 1.1.33

3

Scene Flow & Architecture

 1 Start SceneFADE
TRANSITION 2 Game Scene

Scene Transition System

The `SceneTransitionManager` singleton orchestrates scene changes with a fade-to-black effect using `FadeScreen`. The fade screen renders a full-screen material whose alpha is animated along an `AnimationCurve`, with configurable color and duration. Two modes are available:

- **Synchronous** — `GoToScene(index)` : fade out → wait → load scene
- **Asynchronous** — `GoToSceneAsync(index)` : fade out → load in background → activate when ready

Start Scene Build Index 0

The main menu scene managed by `GameStartMenu`. Contains three UI panels:

- **Main Menu** — Start, Options, About, Quit buttons
- **Options** — Volume slider (`AudioListener.volume`) + Turn Type dropdown (Snap vs Continuous, persisted via `PlayerPrefs("turn")`)
- **About** — Project credits and information

Game Scene Build Index 1

The main gameplay scene containing the vehicle, battery compartment, OBD scanner, diagnostic UI, and all interactive elements. Controlled by `BatteryDiag` — the central gameplay controller.

0

Safety Equipment

Put on gloves and safety glasses. Grabbing gloves changes hand material color; donning glasses activates a "worn" visual on the player. Both tasks must be complete before advancing.

1

Open Battery Latch

An arrow hint appears pointing to the battery latch. The `OpenLatchUI` panel guides the user through releasing the latch to access the battery compartment.

2

Visual Inspection

Three toggle groups: **Casing**, **Terminal**, and **Leakage**. Each locks after selection. Vehicle exterior toggable (Full / Half / No Exterior) via `ToogleView`.

3

Connect OBD Scanner

Scanner and wire appear on a table. User grabs the scanner and brings it to the OBD port (trigger zone via `ColliderManager`). On connection, hover model swaps to "connected" state.

4

Read Diagnostic Data

OBD screen activates showing diagnostic parameters. Reading is optional — Next button is immediately available, but individual read statuses are tracked with checkmarks.

5

Quiz & Results

Multiple-choice battery condition questions. Correct → green indicator with explanation. Incorrect → prompts review. **Finish** compiles all results to a clipboard summary.

5.1 Core Gameplay Scripts [Assets/Script/](#)

SCRIPT	ROLE	KEY DETAILS
BaterryDiag	Central gameplay controller	Manages 6-step procedure, task tracking via TaskGroup[] , UI page switching, Next/Prev logic, step initialization, and final result compilation on the clipboard.
ToogleGroup	Inspection answer manager	Manages three toggle groups (casing, terminal, leakage). Validates answers, locks groups after selection, saves answers to BaterryDiag.InspectionAnswer[] .
ColliderManager	Trigger-zone interaction handler	Detects when objects enter trigger zones. Plays audio on success, marks tasks done, resets objects to original positions. Communicates with BaterryDiag via TaskScenario / IndexTaskGroup / TaskIndex .
ComponentConfig	Inspectable component behavior	Toggles Outline (QuickOutline) highlighting on/off and shows a "look-at-camera" UI hover panel that always faces the player's headset.
ToggleHover	Tooltip display	Shows/hides a hover canvas with TextMeshPro tooltip text on pointer enter/exit events (IPointerEnterHandler / IPointerExitHandler).
ToggleView	Vehicle exterior visibility	Toggles between Full Exterior, Half Exterior, and No Exterior views for inspection purposes.
slideShowManager	Slide navigation	Generic previous/next page navigator using a sprite array displayed on an Image component. Handles button enable/disable at boundaries.
CiihuyCurvedMesh	Procedural mesh generation	Creates curved meshes along Catmull-Rom spline paths. Works in both editor ([ExecuteAlways]) and runtime.
sceneLoader	Scene loading utility	Simple scene loading with async support and application quit functionality.
switchMaterial	Material switching stub	Placeholder for material-switching functionality (currently empty implementation).

5.2 Core Systems Scripts [Assets/Scripts/](#)

SCRIPT	ROLE	KEY DETAILS
<code>AudioManager</code>	Audio system singleton	<code>DontDestroyOnLoad</code> singleton. Serializable <code>Sound</code> class (name, clip, volume, pitch, loop). Plays sounds by string name: <code>Play("name")</code> . Credits: Brackeys tutorial pattern.
<code>UIAudio</code>	UI audio hooks	Attaches to UI elements to play audio on pointer click, enter, and exit events via the AudioManager.
<code>SceneTransitionManager</code>	Scene switching singleton	Fade-to-black scene transitions. Supports sync and async loading. Works with <code>FadeScreen</code> for the visual fade effect.
<code>FadeScreen</code>	Full-screen fade renderer	Animates a material color property along an <code>AnimationCurve</code> . Configurable color, duration, and auto-fade-in on start.
<code>GameStartMenu</code>	Main menu controller	Manages Main Menu / Options / About panel visibility. Wires button onClick events. Triggers async scene transition on Start.
<code>SetOptionFromUI</code>	Options binding	Binds volume slider to <code>AudioListener.volume</code> and turn-type dropdown to <code>PlayerPrefs("turn")</code> .
<code>SetTurnTypeFromPlayerPref</code>	Locomotion type switcher	Enables/disables <code>ActionBasedSnapTurnProvider</code> vs <code>ActionBasedContinuousTurnProvider</code> based on stored PlayerPrefs.

5.3 Third-Party & Sample Scripts

SCRIPT	SOURCE	ROLE
<code>Outline</code>	QuickOutline	Mesh-based outline rendering with 5 modes (OutlineAll, Visible, Hidden, Silhouette, etc.)
<code>AnimateHandOnInput</code>	Oculus Hands	Drives hand animation via controller input (grip, trigger)
<code>Moveliftcar</code>	Lift Car asset	Keyboard-controlled lift table movement between forward/rear positions
<code>Door</code>	XRI Examples	Physics-based hinged door interaction
<code>XRKnob</code> / <code>XRSlider</code> / <code>XRLever</code> / <code>XRPushButton</code> / <code>XRJoystick</code> / <code>XRGripButton</code>	XRI Examples	3D UI controls for diegetic VR interfaces
<code>Walkthrough</code> / <code>WalkthroughStep</code>	XRI Examples	Guided VR walkthrough system with step triggers

6.1 Task Tracking System

The `TaskGroup` serializable class is the backbone of progress tracking:

```
public class TaskGroup {
    public string TaskName;           // Display name for the step
    public bool[] TaskStatus;         // Per-task completion flags
    public Image[] TaskResultImage;   // Checkmark images for results clipboard
}
```

C#

How it works: Each step has a `TaskGroup` with N subtasks. As the user completes subtasks (via `ColliderManager` triggers, `Step0()`, `Step3()`, `Step4()`), `TaskStatus[i]` is set to `true`. `checkTask()` compares completed tasks against the total — when all are done, the Next button becomes interactable.

6.2 Audio System

`AudioManager` is a `DontDestroyOnLoad` singleton that persists across scene loads. `Sound` objects are configured in the Inspector with name, clip, volume, pitch, and loop settings. Playback is by string name lookup:

```
AudioManager.instance.Play("click_sound");
```

C#

`UIAudio` hooks Unity UI pointer events (`IPointerClickHandler`, `IPointerEnterHandler`, `IPointerExitHandler`) to automatically trigger named audio clips on UI interaction.

6.3 VR Locomotion

Two turn providers from XR Interaction Toolkit:

- **Snap Turn** — `ActionBasedSnapTurnProvider` — rotates the player in fixed angular increments
- **Continuous Turn** — `ActionBasedContinuousTurnProvider` — smooth rotation

The user's preference is stored in `PlayerPrefs("turn")` and applied at startup by `SetTurnTypeFromPlayerPref`, which enables one provider and disables the other.

6.4 Interaction System

Trigger Zones — `ColliderManager` uses `OnTriggerEnter` to detect when tagged objects enter a zone. Configurable via `CollideWithTag`. Supports multiple scenarios through `TaskScenario` branching.

Outline Highlighting — `ComponentConfig` toggles QuickOutline's `Outline` component on inspectable objects, with a companion UI hover panel that billboards toward the player's headset.

Tooltips — [ToggleHover](#) shows TextMeshPro tooltip panels on pointer hover.

3D UI Controls — XRI Examples provide knobs, sliders, levers, push buttons, joysticks, and grip buttons for diegetic VR interfaces.

7 3D Assets & Resources

ASSET	PATH	CONTENTS
 Lift Car	Assets/Lift Car/	Car lift table (FBX model, PSD textures, prefab) + Moveliftcars.cs for keyboard-driven lift movement
 Scissor Lift	Assets/3D/Scissor Lift Table/	Scissor lift OBJ model with texture maps
 Furniture	Assets/Tables and Chairs/	10 prefabs (5 tables + 5 chairs) with PBR texture maps (height, metallic, normal, roughness) + sample scene
 Oculus Hands	Assets/Plugins/Oculus Hands/	Left/right hand model prefabs with AnimateHandOnInput script for controller-driven animation
 QuickOutline	Assets/QuickOutline/	Chris Nolet's mesh outline system — shaders, materials, Outline.cs component + sample scene
 XRI Examples	Assets/XRI_Examples/	Reference implementations: physics doors, socket interactors, 3D UI controls, hover highlights, walkthrough system, props + comprehensive demo scene
 XRI Starter Assets	Assets/Samples/XRI Toolkit/2.3.2/	Preconfigured XR Origin, teleportation, interactors, device simulator, and UI samples
 Kenney Audio	Assets/Audio/Kenney Audio/	4 CC0-licensed audio packs: digital, interface sounds, sci-fi, UI audio (130+ clips)

8 Project Folder Structure

```
VR-Electric-Vehicle/
├── Assets/
│   ├── Scripts/           ← Core systems (Audio, Menu, Scene, Options)
│   │   ├── AudioManager.cs  · Singleton audio playback
│   │   ├── UIAudio.cs       · UI event → audio hooks
│   │   ├── FadeScreen.cs    · Full-screen fade with AnimationCurve
│   │   ├── SceneTransitionManager.cs · Scene switching with fade
│   │   └── GameStartMenu.cs · Main menu controller
```

- └─ SetOptionFromUI.cs · Volume + turn type binding
- └─ SetTurnTypeFromPlayerPref.cs · Snap vs Continuous turn
- └─ Script/ ← Gameplay mechanics
 - └─ BatteryDiag.cs ★ Central gameplay controller
 - └─ ToggleGroup.cs · Inspection answer validation
 - └─ ColliderManager.cs · Trigger zone interactions
 - └─ ComponentConfig.cs · Outline highlighting + billboard UI
 - └─ ToggleHover.cs · Tooltip display
 - └─ ToggleView.cs · Vehicle exterior visibility
 - └─ slideShowManager.cs · Slide navigation
 - └─ CiihuyCurvedMesh.cs · Spline mesh generator
 - └─ sceneLoader.cs · Scene loading utility
 - └─ switchMaterial.cs · Material switch (stub)
- └─ Scenes/ ← Build scenes + backups
 - └─ 1 Start Scene.unity · Main menu (build index 0)
 - └─ 2 Game Scene.unity · Gameplay (build index 1)
 - └─ Backup *.unity · 7 backup variations
- └─ Prefabs/ ← Project prefabs
 - └─ Fader Screen.prefab
 - └─ Transition Manager.prefab
- └─ 3D/ ← 3D models & textures
- └─ Lift Car/ ← Car lift model + script
- └─ Tables and Chairs/ ← 10 furniture prefabs + PBR textures
- └─ QuickOutline/ ← Outline highlight system
- └─ Plugins/Oculus Hands/ ← Hand models & animation
- └─ Samples/ ← XRI Toolkit starter assets
- └─ XRI_Examples/ ← XRI reference implementations
- └─ UI/ ← Sprites, fonts, TMP assets
- └─ Audio/ ← Kenney CC0 audio packs
- └─ ProjectSettings/ ← Unity project configuration
- └─ Packages/ ← Package manifest
- └─ README.md
- └─ CLAUDE.md

9.1 Build Scenes

The project's build settings reference two scenes in `ProjectSettings/EditorBuildSettings.asset` :

INDEX	SCENE	EDITOR NAME
0	<code>1 Start Scene.unity</code>	Backup 7
1	<code>2 Game Scene.unity</code>	Backup 6

⚠ Build settings reference "Backup 6" and "Backup 7" names. When changing scenes, update `EditorBuildSettings.asset` accordingly.

9.2 Building the Project

1. Open the project in **Unity 2022.3 LTS** (or compatible version)
2. Go to **File** → **Build Settings**
3. Select target platform:
 - **Android** for Meta Quest (IL2CPP, ARM64)
 - **Windows Standalone** for PC VR (Mono, x86_64)
4. Ensure both scenes are in the build list in the correct order
5. Click **Build** (or Build and Run for a connected device)

There is no CI/CD pipeline — builds are made locally through the Unity Editor.

9.3 Define Symbols & Testing

SYMBOL	APPLIED TO
<code>USE_INPUT_SYSTEM_POSE_CONTROL</code>	Android, Standalone, Windows Store Apps

Testing: Run via **Window** → **General** → **Test Runner**. Unity Test Framework 1.1.33 is installed but no pre-existing test scripts exist — PlayMode and EditMode tests would need to be authored for new development.



